핵심 주제 (응모 분야 Living Assist)

방 습도계가 45%여도 차가운 벽 표면은 80%임

벽 표면 습도를 계산해 곰팡이 임계 전에 가습기와 선풍기를 스스로 조절하는 Matter 디바이스를 제안함

실내 습도 (방 습도계)

45%

방 공기 기준 값

12°C 북측 벽 표면

80%

ISO 13788 곰팡이 임계

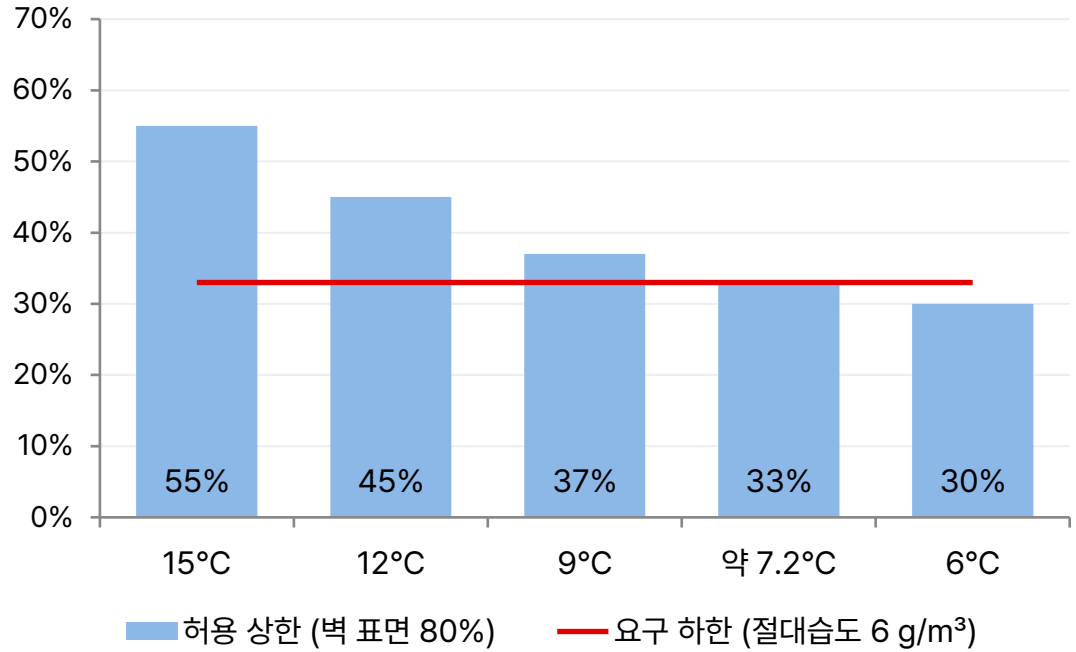
서로 충돌하는 침실의 세 목표

① 수면: 침실 18~20°C
사용자가 정하는 제약 조건임 (Prather, SLEEP 2015)

② 점막 방어: 절대습도 6 g/m³ 이상
노드가 제어함 (Shaman & Kohn, PNAS 2009)

③ 곰팡이와 진드기: 벽 표면 80% 이하
노드가 제어함 (ISO 13788:2012)

벽 표면 온도별 허용 실내 습도 (실내 21°C)



벽이 약 7.2°C 아래로 내려가면 두 목표를 함께 만족하는 구간이 사라짐.
노드는 선풍기로 벽 표면을 데워 이 구간을 되찾음.

주요 개발 내용 ① 시스템 구성과 회로

보드 안에서 판단하고, 릴레이로 개입하고, Matter로 보고함

측정
실내 온도와 습도
벽 표면 온도, 침구 습도
시연에서는 가상 침실이 값을 생성함

판단
nRF54LM20 DK 안에서 처리함
표면 습도 계산, 75/70% 판정
학습과 예측, 클라우드 불필요

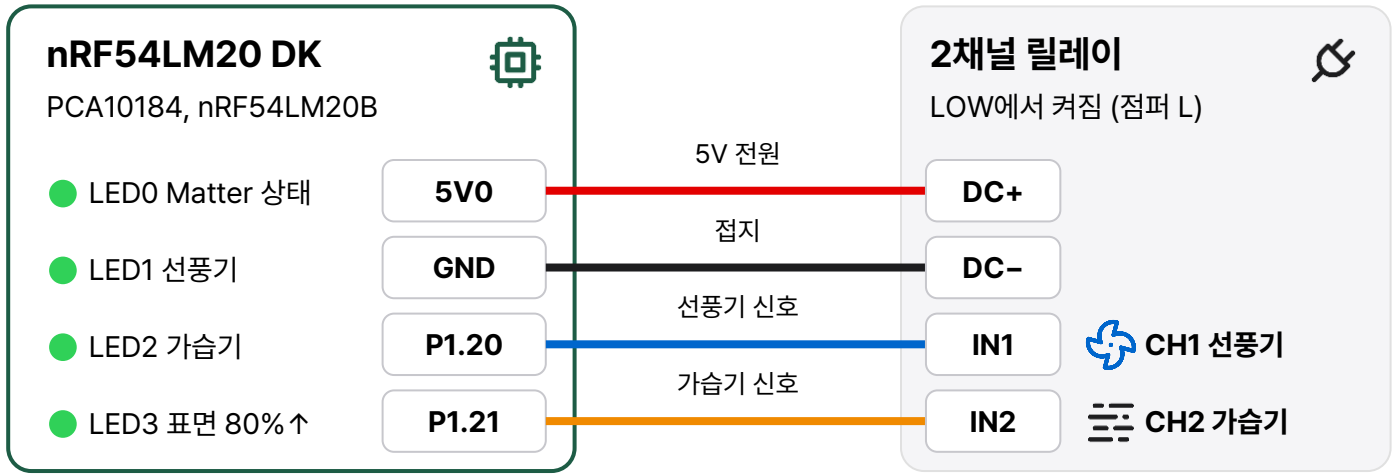
개입
2채널 릴레이
CH1 선풍기, CH2 가습기
GPIO P1.20, P1.21

노출 (Matter)
EP1 선풍기, EP5 가습기
EP2, EP3 습도, EP4 온도
표준 클러스터 사용

SmartThings Station과 앱
Thread로 보고, 수신 확인

닫힌 고리: 선풍기와 가습기가 바꾼 환경이 다시 측정값으로 돌아옴

회로: DK ↔ 2채널 릴레이



하드웨어 구현에서 해결한 것

보드 타깃 nrf54lm20b
칩이 nRF54LM20B라 설명서의 a 타깃 대신 b로 빌드함

devicetree overlay로 릴레이 핀 정의
P1.20, P1.21을 GPIO_ACTIVE_LOW로 정의함

P1.22 제외
DK의 LED0 핀과 겹쳐 오동작해 옆 핀 P1.21로 이동함

주요 개발 내용 ② Matter SDK 활용

nRF Connect SDK로 노드 하나를 엔드포인트 6개로 직접 구성함

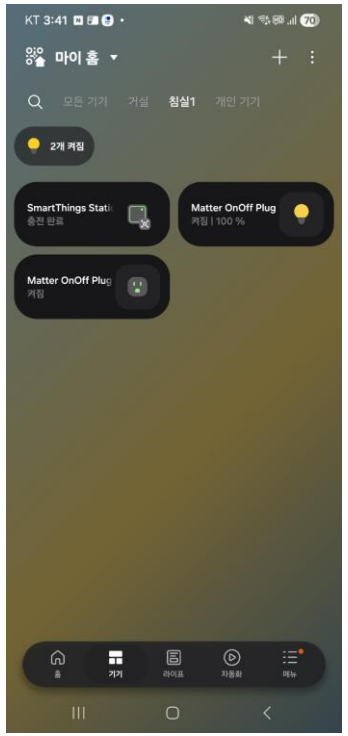
EP	디바이스 타입	표준 클러스터	노드가 하는 일
0	Root Node	Basic Info, 커미셔닝, Thread	커미셔닝, Thread 합류
1	Dimmable Light → 선풍기	On/Off, Level, Identify	On/Off → 릴레이 CH1
2	Humidity Sensor	Relative Humidity Meas.	벽 표면 습도 (노드 계산)
3	Humidity Sensor	Relative Humidity Meas.	실내 습도 (방 습도계)
4	Temperature Sensor	Temperature Measurement	벽 표면 온도
5	On/Off Plug-in → 가습기	On/Off, Groups	On/Off → 릴레이 CH2

ZAP 데이터 모델
light_bulb.zap에 EP2~EP5를 추가하고 west zap-generate로 생성함

속성 게시
Accessors로 값 설정, ScheduleWork로 Matter 스레드에서 실행, 허브에 ReportData 전송함

앱 조작 수신
MatterPostAttributeChangeCallback으로 받아 릴레이 구동과 판단부에 반영함

커미셔닝
공장 데이터와 BLE로 SmartThings에 등록하고 Thread 네트워크에 합류함



SmartThings 등록 화면

11 / 11
허브 보고 / 수신 확인 (시연 1회)

92 ms
보고 후 허브 응답 (평균)

42%
Flash 사용 (RAM 36%)

v3.4.1
nRF Connect SDK

SmartThings 앱은 스위치와 센서가 섞인 기기에서
센서 엔드포인트를 표시하지 않음.
그래서 EP2~EP4 값은 시연 화면에서 함께 보여줌.

주요 개발 내용 ③ 온디바이스 판단

노드가 스스로 계산하고 학습하고 개입함

계산 (0.5초마다)

포화 수증기압 (Magnus 식, hPa)

$$P_S(T) = 6.112 \times \exp\left(\frac{17.62T}{243.12 + T}\right)$$

벽 표면 상대습도

$$RH_{표면} = RH_{실내} \times \frac{P_S(T_{실내})}{P_S(T_{벽})}$$

절대습도 (g/m³)

$$AH = \frac{216.7 \times RH \times P_S(T)}{100 \times (273.15 + T)}$$

허용 상한 (표 3)

$$RH_{상한} = 80 \times \frac{P_S(T_{벽})}{P_S(T_{실내})}$$

판단, 학습, 예측을 모두 DK 안에서 처리함.
Wi-Fi나 클라우드가 끊겨도 방어가 계속됨.

A 곰팡이 방어
벽 표면 75%에서 가습을 멈추고 70% 미만에서만 재개함 (임계 80%에 여유 5%p)

B 점막 방어
절대습도 6.5 g/m³ 미만이면 가습하고 8 g/m³에서 정지함

선풍기 개입
가습을 멈춰도 75% 이상이면 0.5초 뒤 선풍기를 켜 벽 표면을 데움

E 학습
선풍기를 켠 뒤 벽 표면 상승폭을 재고(시연 +2.8°C) 끄는 시점을 판단함

E 예측
벽 표면 온도 추세로 임계 80% 도달까지 남은 시간을 계산함

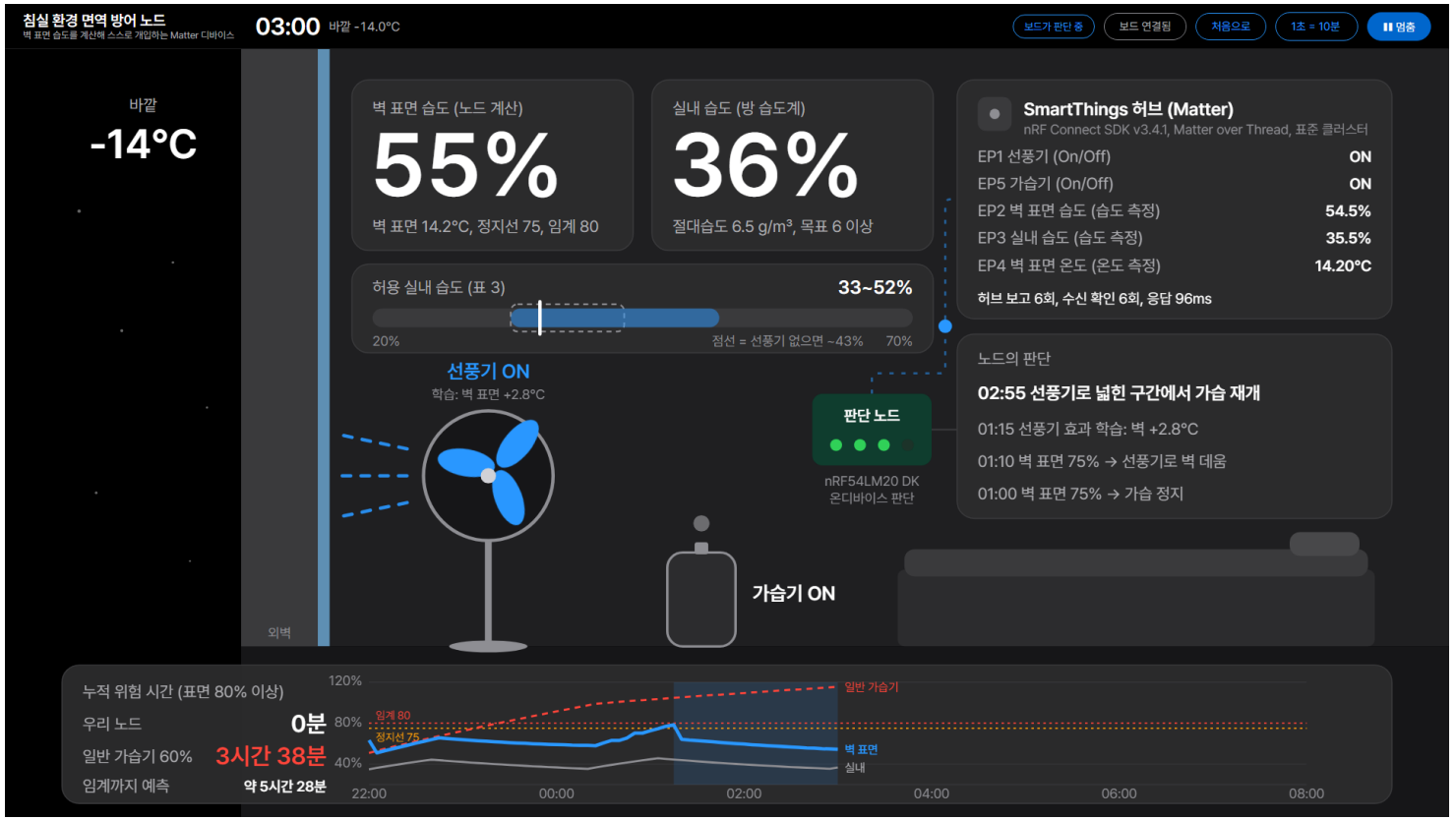
동시 가동
선풍기로 넓힌 구간 안에서 선풍기를 켠 채 가습을 재개함 (②, ③ 동시 충족)

사용자 우선
앱과 버튼 조작은 30분간 존중하고, 벽 표면 80%를 넘으면 방어를 우선함

기록
벽 표면 80% 이상이었던 시간을 누적 위험 시간으로 기록함

시연 내용

한파의 밤 10시간을 60초로 재현해 누적 위험 시간 0분을 달성함



실제 보드를 연결한 03:00 화면. 실내와 벽 표면 습도, 노드의 판단, 가습기와 선풍기, 허브 보고를 한 화면에 보여줌

0분

누적 위험 시간, 우리 노드 (실제 보드)

8시간 38분

같은 밤 일반 가습기(60% 목표)였다면

78% / 119%

최고 벽 표면 습도 (우리 노드 / 일반 가습기)

+2.8°C

노드가 학습한 선풍기 효과

22:00
건조 → 가습 시작

01:00
표면 75% → 가습 정지

01:10
선풍기 ON, 효과 학습

02:55~
선풍기 켠 채 가습

07:25
벽 회복 → 선풍기 OFF